CHAPTER 6

CLIENT SIDE SCRIPTING USING JAVASCRIPT

CHAPTER ESSENCE AT A GLANCE:
JavaScript empowers responsive, intelligent web interfaces through client-side dynamic execution, event-driven architecture, built-in dialog interactions, DOM manipulation, and automated form validation.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Event-Driven Execution and Script Integration:** JavaScript programs execute asynchronously in response to user-triggered browser events like clicks, form submissions, and input modifications, embedded using the script tag in the head or body sections.

• **Dynamically Typed Variables and Operators:** Variables in JavaScript are loosely typed, automatically adapting storage representations at runtime, manipulated through arithmetic, relational, logical, and string concatenation operators.

• **Interactive Dialogs and Data Type Conversions:** Built-in dialog functions alert, confirm, and prompt provide lightweight user interaction, working with conversion functions parseInt, parseFloat, and isNaN to sanitize textual user input into numbers.

• **DOM Access and Client-Side Form Validation:** The Document Object Model organizes HTML elements into accessible object trees, allowing scripts to inspect input fields, enforce validation rules, display inline error messages, and halt invalid form submission using return false.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{Event Dispatch Pipeline} = \text{User Action (Click or Submit)} \longrightarrow \text{Event Handler} \longrightarrow \text{JS Function Execution}$$

Event-Driven Asynchronous Dispatch

$$\text{Dialog Interactions} = \frac{\text{alert}(msg) \rightarrow \text{OK Notice}, \quad \text{confirm}(msg) \rightarrow \text{True or False Boolean}}{\text{prompt}(msg) \rightarrow \text{User String Input or Null}}$$

Standard Browser Dialog Interactions

$$\text{Data Type Conversion} = \text{parseInt}(str) \rightarrow \text{Integer}, \quad \text{isNaN}(val) \rightarrow \textbf{true} \text{ if not a valid number}$$

Numerical Parsing and Sanitization

$$\text{Form Validation Predicate} = \textbf{if } (\text{Field Invalid}) \longrightarrow \text{Display Error; } \textbf{return false;}$$

Client-Side Submission Guard Rule

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** JavaScript is strictly case-sensitive; function names like getElementById and variables like totalScore and TotalScore represent completely distinct identifiers.

• **Alert:** The prompt() function always returns user input as a string; attempting arithmetic addition without using parseInt() or parseFloat() results in string concatenation.

• **Alert:** Always write return validateForm() in the onsubmit attribute of the form tag; writing only validateForm() ignores the boolean return value and submits invalid data.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. What is JavaScript? State two key advantages of client-side validation over server-side validation. [Score: 2 Marks]

Answer & Key Valuation Steps:

• Definition: A lightweight, interpreted, object-based client-side scripting language primarily used to build interactive and dynamic web pages. **[1 Mark]**

• Advantages: Provides immediate visual feedback to users without waiting for a server round-trip, significantly reducing server processing load and conserving network bandwidth. **[1 Mark]**

2. Explain the three built-in dialog boxes in JavaScript: alert(), confirm(), and prompt(). [Score: 2 Marks]

Answer & Key Valuation Steps:

• alert(message): Displays an informational message box with an OK button to alert the user; returns no value. **[0.75 Mark]**

• confirm(message): Displays a verification dialog with OK and Cancel buttons; returns boolean true if OK is clicked, false if Cancel is clicked. **[0.75 Mark]**

• prompt(message, defaultText): Displays an input dialog allowing the user to enter a text string; returns the entered string or null if cancelled. **[0.5 Mark]**

3. Explain isNaN() and parseInt() functions with illustrative JavaScript code examples showing how they prevent invalid arithmetic. [Score: 3 Marks]

Answer & Key Valuation Steps:

• parseInt(str): Converts a string representation of digits into an integer; stops reading at the first non-numeric character. Example: parseInt('45') yields integer 45. **[1 Mark]**

• isNaN(value): Evaluates whether a value is Not-a-Number; returns true if the value cannot be parsed as a valid numeric number, and false if numeric. Example: isNaN('abc') returns true. **[1 Mark]**

• Sanitization Pattern: if (isNaN(val)) alert('Please enter numeric value'); else total = parseInt(val) + 10; ensures robust calculation. **[1 Mark]**

4. Explain event handling in JavaScript. Describe the events onclick, onchange, and onsubmit with sample usage. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Event Concept: Events are actions occurring in the browser (such as clicking a button or pressing a key) that trigger associated JavaScript handler functions. **[1 Mark]**

• onclick & onchange: onclick triggers when a user clicks an element (e.g. <button onclick='show()'>); onchange fires when an input control loses focus after its value changed (e.g. dropdown selection). **[1 Mark]**

• onsubmit: Fires when a form submit button is clicked; writing onsubmit='return validate()' allows cancelling submission if validation fails. **[1 Mark]**

5. Write a JavaScript function that takes a number as input and displays whether the number is positive, negative, or zero using an alert box. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Function Header and Parsing: function checkNumber() { var n = parseInt(prompt('Enter a number:')); **[1 Mark]**

• Conditional Branches: if (n > 0) alert('Positive Number'); else if (n < 0) alert('Negative Number'); **[1 Mark]**

• Zero and Closure: else if (n == 0) alert('Zero'); else alert('Invalid Input'); } **[1 Mark]**

6. Write a JavaScript program to validate a password input field, ensuring the entered password has a minimum length of 8 characters. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Function Definition: function validatePassword() { var pwd = document.getElementById('pass').value; **[1 Mark]**

• Length Check: if (pwd.length < 8) { alert('Password must be at least 8 characters long!'); return false; } **[1 Mark]**

• Success Branch: return true; } **[1 Mark]**

7. Write complete HTML and JavaScript code to validate a registration form, ensuring the Name field is not empty, the Age field contains a number between 18 and 60, and displaying appropriate alerts. [Score: 4 Marks]

Answer & Key Valuation Steps:

• HTML Form Setup: <form name='regForm' onsubmit='return validate()' action='save.php'> Name: <input type='text' id='userName'>
 Age: <input type='text' id='userAge'>
<input type='submit' value='Submit'></form> **[1 Mark]**

• Name Validation: function validate() { var name = document.getElementById('userName').value.trim(); if (name == '') { alert('Name cannot be empty'); return false; } **[1 Mark]**

• Age Validation: var age = parseInt(document.getElementById('userAge').value); if (isNaN(age) || age < 18 || age > 60) { alert('Age must be a valid number between 18 and 60'); return false; } **[1 Mark]**

• Form Submission Approval: return true; } ensures invalid submissions are prevented before network transmission. **[1 Mark]**